



Module 2 Assignment Data driven approach toward Stock investing

Module 2 Assignment Project—NTU

Agenda

1. Executive summary
2. User Requirement
3. Tools & Technologies
4. Pipeline Architecture
5. Virtual Machine
6. Data Ingestion
7. Data Warehouse Design
8. ELT & Data Quality
9. Pipeline Orchestration



Executive summary

- Our team has identify using Datasource from Yahoo Finance & Fred (Federal Reserve Economic Data) to extract financial intelligent for Tech Stock investor
- Using our skill sets we learned from the course we will attempt to make insight with dataset we have available
- We have choose to run this project on Google Cloud Virtual machine, Big Query, Dagster, DBT and Great expectation module.





DISCLAIMER

This project is for educational and entertainment purposes only.

The analyses, charts, and commentary provided herein are part of an academic data science project. They are **not intended as financial, investment, or trading advice.**

Although real financial data is used, the information and models presented **do not guarantee accuracy, completeness, or future performance.**

The author(s) of this project are **not licensed financial advisors.** Any investment-related observations are purely hypothetical and for illustrative purposes.

Please do not use this content as a basis for investment decisions. Always seek advice from a qualified financial professional before making any financial decisions.

User Analysis story 1



Sector Rotation Analysis Using ETF Data (2020–2025)

1. 🎯 Objective of the Analysis

- Visualize **sector rotation** across economic cycles.
- Track sector performance using **sector ETFs**.
- Identify patterns in sector leadership around **key macro events** (e.g., Fed decisions, inflation).
- Support portfolio decisions with **data-driven insights**.

2. 📖 What Is Sector Rotation?

- Sector rotation refers to the **cyclical movement** of capital among sectors during different economic phases.
- Investors shift between sectors based on:
 - Growth expectations
 - Interest rates
 - Inflation trends

Textbook rotation sequence:

Cycle Phase	Leading Sectors
Early Recovery	`Technology`, `Consumer Discretionary`, `Communication Services`
Mid Expansion	`Industrials`, `Materials`, `Energy`
Late Expansion	`Consumer Staples`, `Health Care`, `Utilities`
Recession	`Financials`, `Real Estate`

3. 🛠️ Tools & Data

- ETFs: Sector proxies like `XLK` (Technology), `XLF` (Financials), etc.
- Data Source: **[Yahoo Finance]**(<https://finance.yahoo.com/>) via `yfinance` Python package.
- Timeframe: **`2019-06-01` to `2025-06-01`**
- Data Handling: **`pandas`, `matplotlib`, `seaborn`**
- Data Source (alternative): Retrieve ETF tickers dynamically from BigQuery.

4. 📊 Methodology

📌 a. Monthly Returns & 3-Month Rolling Cumulative Returns

- Resample ETF closing prices to **monthly**.
- Compute **percentage change** for monthly returns.
- Apply a **3-month rolling product** for cumulative trends.

📌 b. Rolling Heatmap of Returns

- `seaborn.heatmap` to display rolling 3-month returns.
- Color-coded to identify hot and cold sectors over time.

📌 c. Sector Ranking Heatmap

- Rank sectors each month by performance (`1 = Best`).
- Reveals **sector leadership rotation** visually.

5. 🧠 Key Insights

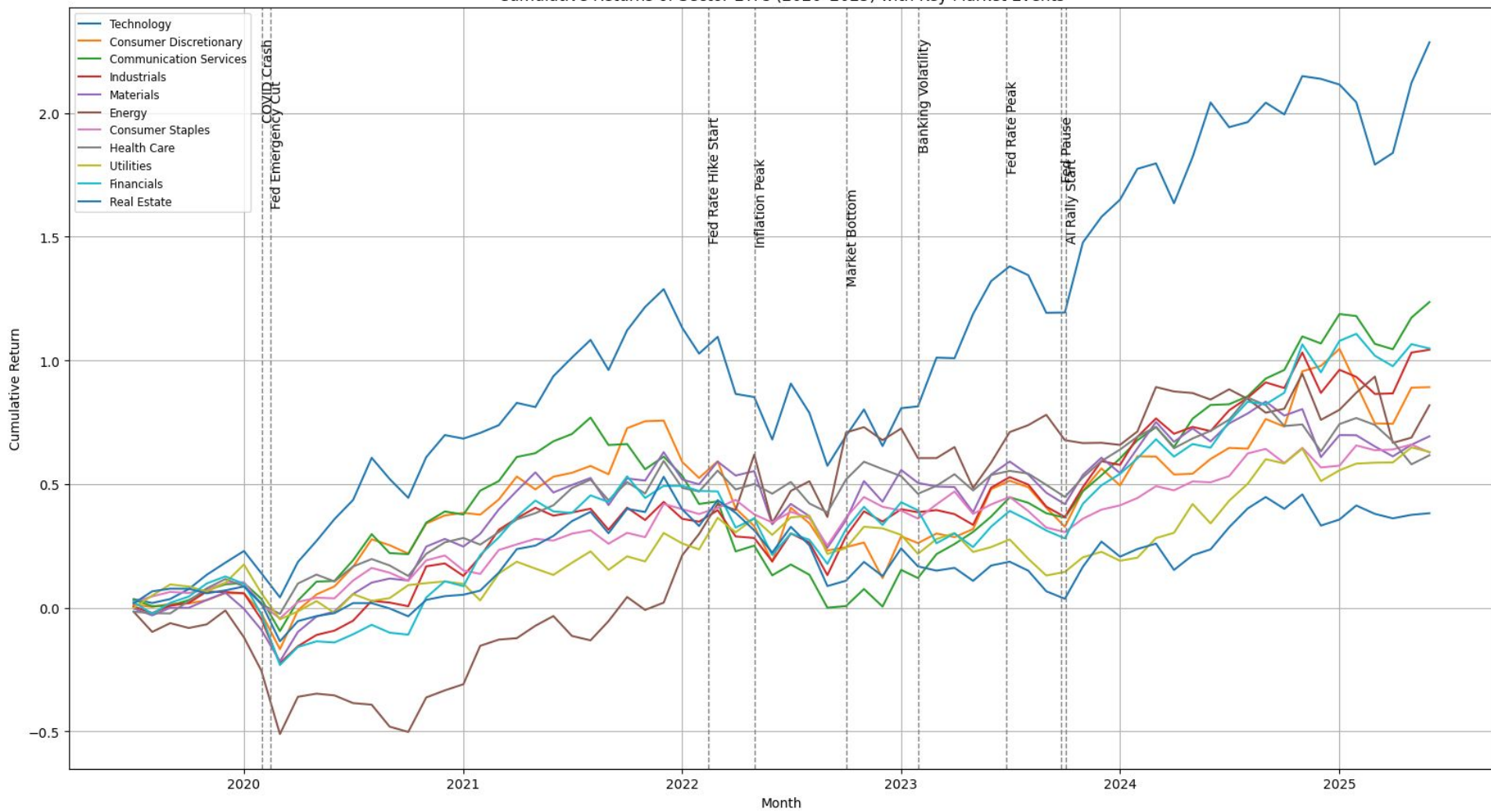
Event	Observation
----- -----	
COVID Crash (Mar 2020)	All sectors dropped; Tech rebounded fastest
Fed Rate Hike Cycle (Mar 2022)	Defensive sectors (Staples, Health Care) outperformed
Inflation Peak (Jun 2022)	Market bottom followed in Oct 2022
AI Rally (Nov 2023)	Tech, Comm Services resurged strongly
Utilities & Staples	Provided downside protection during volatility

- The **heatmaps and plots show clear sector rotation** aligned with macro phases.
- Defensive sectors shine in uncertainty; cyclicals lead in growth phases.

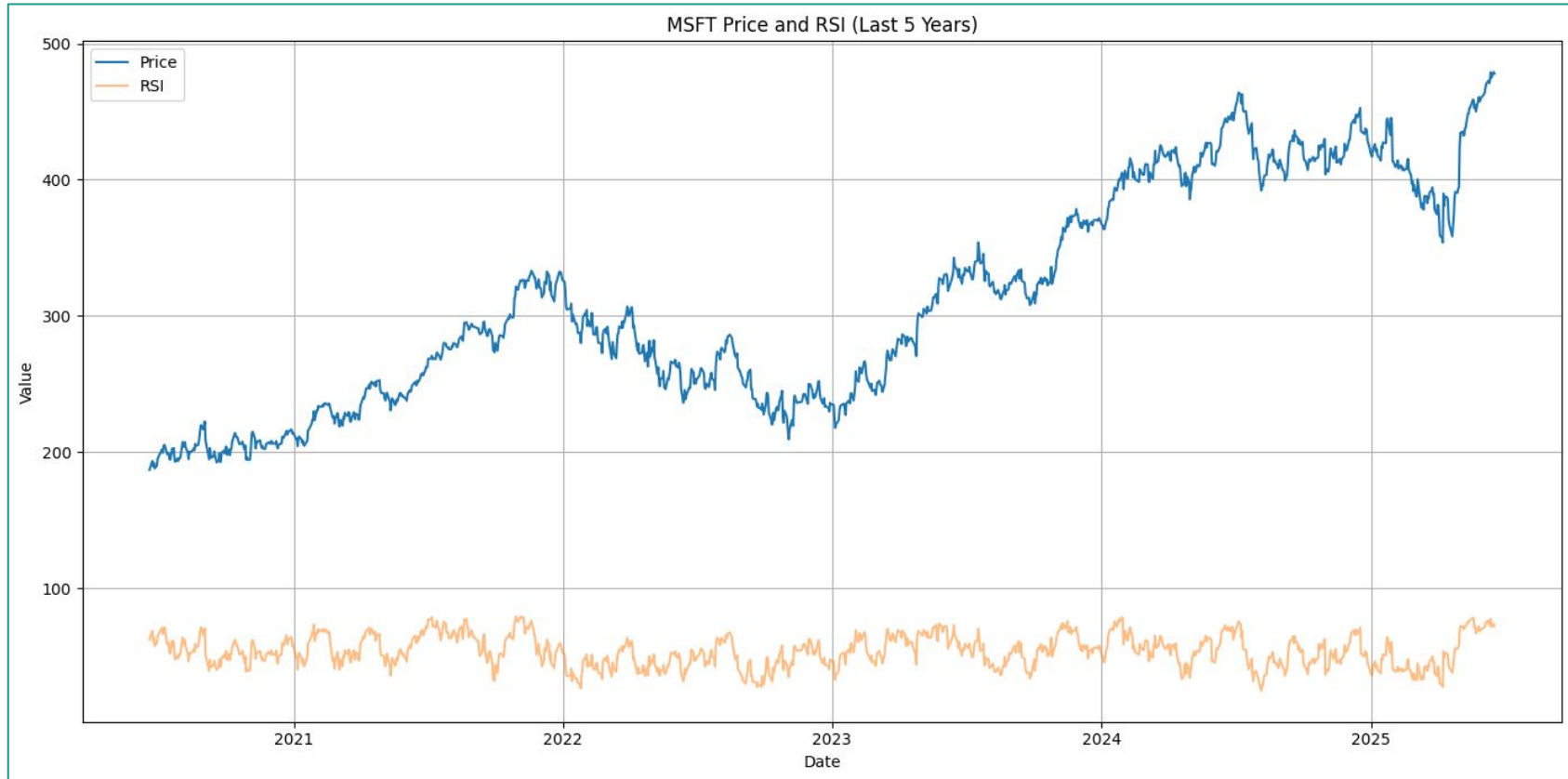
6. 🧠 Real-World Applications

- 🧭 **Portfolio Strategy:** Align with top-performing sectors per cycle.
- 🛡️ **Risk Management:** Avoid sectors hurt by rate hikes or inflation.
- 📅 **Macro Timing:** Anticipate rotation using economic indicators (e.g., CPI, FOMC).

Cumulative Returns of Sector ETFs (2020-2025) with Key Market Events

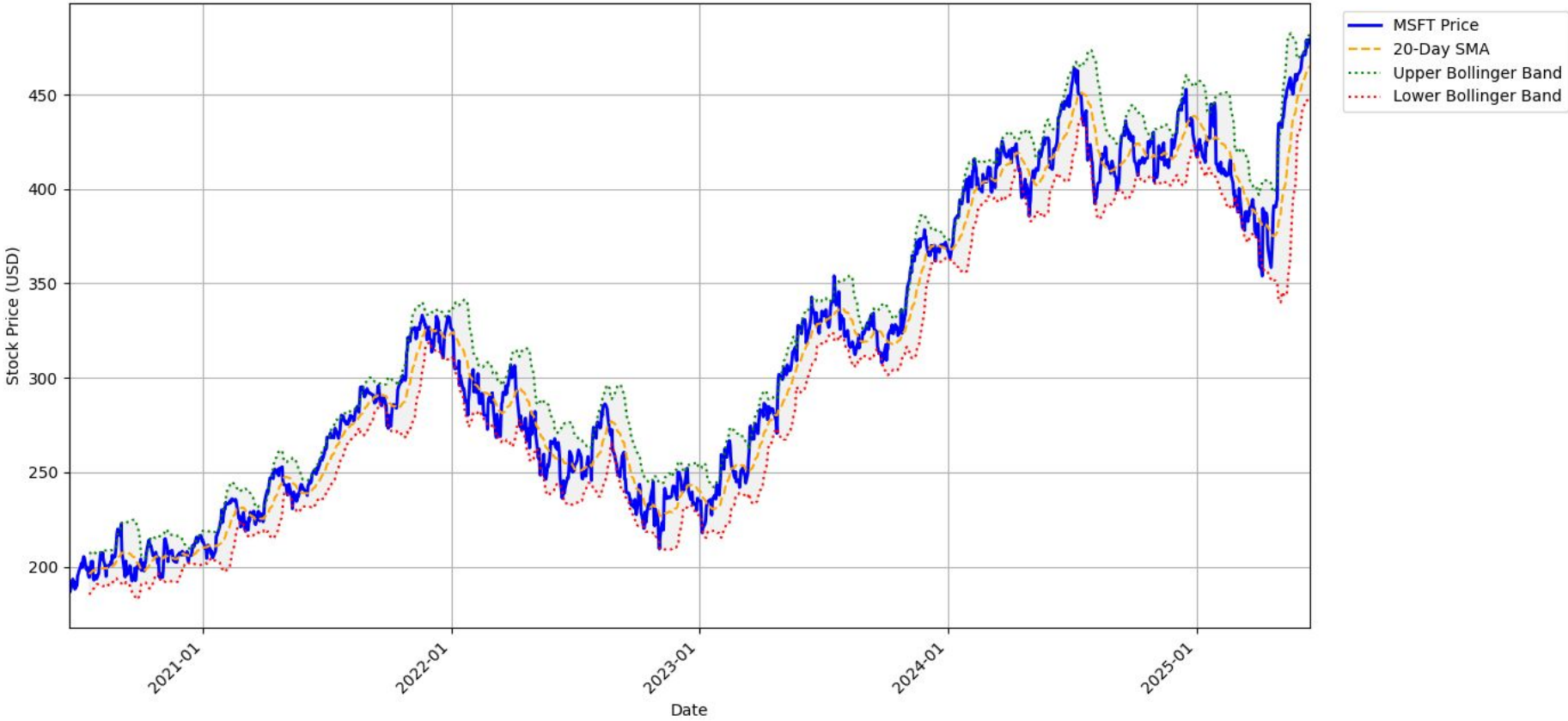


Specific stock analysis





MSFT (Technology) Price Trend with Bollinger Bands
2020-06-17 to 2025-06-17



Finding



Plotting data from 2020-06-17 to 2025-06-17

Price range: \$186.97 to \$479.14

=== Detailed Analysis: MSFT (Technology) ===

Current Price: 478.04

P/E Ratio: 36.91

Dividend Yield: 0.00%

RSI: 72.22

50-Day SMA: 429.82

Valuation Assessment:

- P/E ratio is moderate, compare to sector average
- No dividend yield
- RSI above 70 indicates overbought condition
- Price above 50-Day SMA (positive trend)

Bollinger Bands Interpretation:

- Price is within Bollinger Bands: Neutral range

Recommendation:

Summary:

MSFT trades at a relatively high P/E ratio, which may indicate growth expectations are already priced in., does not offer a dividend, which may reduce its income appeal., is currently overbought based on RSI, suggesting caution., is trading above its 50-day moving average, indicating positive price momentum., is trading within its Bollinger Bands, indicating a balanced technical position.

Consider further analysis of:

- Sector comparison metrics
- Recent earnings reports
- Company fundamentals
- Macroeconomic conditions affecting this sector

Detailed Evaluation: MSFT

⚠ This stock does not appear significantly undervalued:

⚠ Suggestion:

- P/E ratio of 36.91 is near or above sector average (37.96)
- P/B ratio of 11.04 may indicate overvaluation
- RSI of 72.22 may indicate overbought condition

Finding criteria

P/E Ratio vs Sector Average

- **Comparison:**
 - If stock's P/E < 80% of sector average, it is flagged as **undervalued**.
 - *Explanation:* A P/E ratio significantly below the sector average suggests the stock may be underpriced relative to its peers.
 - If stock's P/E ≥ 80% of sector average, it is flagged as a **caution**.
 - *Explanation:* The stock is fairly valued or potentially overvalued compared to the sector.
 -

P/B Ratio (Price-to-Book Ratio)

- **Threshold:**
 - **P/B < 3:**
Considered to reflect a **reasonable valuation**.
 - *Explanation:* Lower P/B ratios are generally seen as safer and potentially undervalued, particularly for asset-heavy companies.
 - **P/B ≥ 3:**
Potential **overvaluation** warning.
 - *Explanation:* Higher P/B ratios may indicate that the stock price is high relative to its book value, which could be a risk unless strongly supported by growth.

Dividend Yield

- **Thresholds:**
 - **> 2.5%:**
Attractive dividend yield → added to **undervaluation reasons**.
 - *Explanation:* Higher dividends can be a strong incentive for income-focused investors.
 - **0% – 2.5%:**
Modest dividend yield → added to **caution list**.
 - *Explanation:* The dividend is present but not particularly compelling.
 - **0%:**
No dividend → not flagged directly in this code, but would naturally reduce the income appeal.

RSI (Relative Strength Index)

- **Thresholds:**
 - **RSI < 40:**
Potential **oversold condition** → added to undervaluation reasons.
 - *Explanation:* Indicates that selling pressure might be overdone, providing a potential buying opportunity.
 - **RSI > 60:**
Potential **overbought condition** → added to caution list.

Finding criteria



P/E Ratio (Price-to-Earnings Ratio)

- **Indicator of Valuation:**
 - **P/E < 15:**
Considered potentially undervalued compared to peers. May suggest the stock is attractively priced relative to its earnings.
 - **P/E 15 - 25:**
Considered reasonably valued. Further comparison to the sector is advised to confirm if it is a good opportunity.
 - **P/E > 25:**
Considered relatively expensive. This could indicate that the market has high growth expectations for the stock, and it may already be fully priced.

Price vs. 50-Day SMA (Simple Moving Average)

- **Indicator of Trend Direction:**
 - **Price > 50-Day SMA:**
Positive price momentum. Suggests a bullish (upward) trend.
 - **Price < 50-Day SMA:**
Possible near-term weakness. Could indicate the stock is losing momentum or is potentially undervalued.

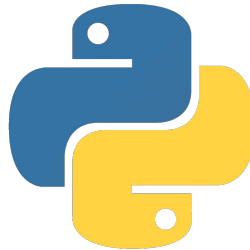
Bollinger Bands

- **Indicator of Volatility and Potential Reversal:**
 - **Price < Lower Band:**
Possible oversold signal. Price may rebound upwards.
 - **Price > Upper Band:**
Possible overbought signal. Price may correct downwards.
 - **Price within Bands:**
Neutral condition. The price is within normal trading volatility.

Tools & Technologies



dagster



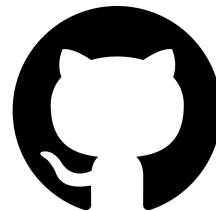
python™



Google Cloud



Google
Big Query

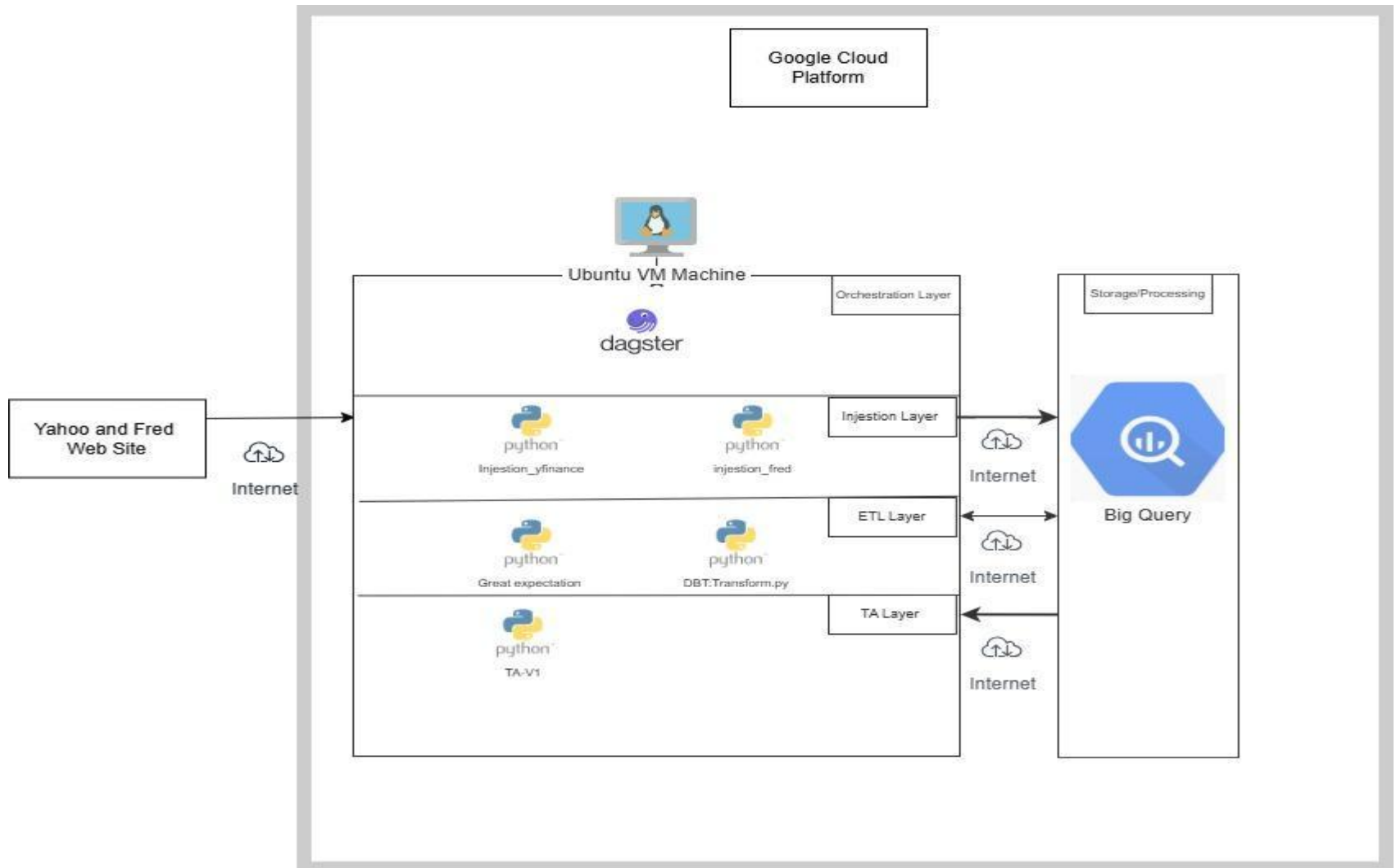


GitHub



dbt

Pipeline Architecture



VM

VM Specifications

- 2 vCPUs
- 10GB HDD
- 4GB RAM

Setup Process (High-Level)

1. Launch VM Instance:
 - Use GCP Compute Engine to create the VM.
 - Upload `create-vm.sh` and run it to configure VM.
2. Prepare Environment:
 - Create a project directory.
 - Upload `environment.yml` and `setup.sh`.
 - Run `setup.sh` to initialize services.
3. Configure Python and GitHub:
 - Create a Conda environment (`environment.yml`).
 - Generate SSH keys and configure GitHub access.
 - Clone the repository.
4. Final Configuration:
 - Enable RDP desktop environment (`xfce4`) for access.
 - Upload BigQuery service key.
5. Run Dagster:
 - Navigate to the ingestion directory and start Dagster.
 - Access the UI via RDP and Chrome.

```
1  #!/bin/bash
2
3  # ---- CONFIGURABLE ----
4  VM_NAME="vm-elt-2"
5  ZONE="us-central1-b"
6  MACHINE_TYPE="e2-highcpu-8"
7  IMAGE_FAMILY="ubuntu-2204-lts"
8  IMAGE_PROJECT="ubuntu-os-cloud"
9  DISK_SIZE="20GB"
10
11 # ---- EXECUTE ----
12 echo "Creating VM: $VM_NAME in $ZONE..."
13
14 gcloud compute instances create $VM_NAME \
15   --zone=$ZONE \
16   --machine-type=$MACHINE_TYPE \
17   --image-family=$IMAGE_FAMILY \
18   --image-project=$IMAGE_PROJECT \
19   --boot-disk-size=$DISK_SIZE \
20   --boot-disk-type=pd-standard \
21   --tags=http-server,https-server \
22   --metadata=startup-script='#!/bin/bash
23     sudo apt update
24     sudo apt install -y nginx' \
25   --scopes=https://www.googleapis.com/auth/cloud-platform
26
27 echo "✅ VM $VM_NAME created successfully."
```

Data Ingestion



The Federal Reserve Economic Data (FRED) is a free online database provided by the Federal Reserve Bank of St. Louis. It offers access to over 800,000 economic time series datasets from sources like U.S. government agencies, international organizations, and central banks. These datasets cover a wide range of topics including GDP, inflation, interest rates, employment, consumer sentiment, and financial market indicators. FRED provides easy-to-use APIs, charts, and downloadable files to help users explore and analyze economic trends. It is widely used by economists, analysts, researchers, and students to track macroeconomic conditions and build data-driven insights.



yfinance is a popular Python library that allows users to easily download historical market data from Yahoo Finance. It provides access to a wide range of financial information including stock prices, company fundamentals, dividends, earnings, and market indices. The library is widely used for financial analysis, backtesting trading strategies, and building investment models because it is simple, fast, and does not require an API key. yfinance also supports features like fetching intraday data, adjusting for splits/dividends, and downloading data for multiple tickers at once. It is commonly used by investors, analysts, data scientists, and students for both educational and professional projects.

Data Ingestion

```
### section here is to download yfinance jacob's code this is usable.  
import yfinance as yf  
import pandas as pd  
from google.cloud import bigquery  
import json  
import os
```

Yfinance is offered as a Python library instead of API endpoint. All extraction of data will need to be via a Python.

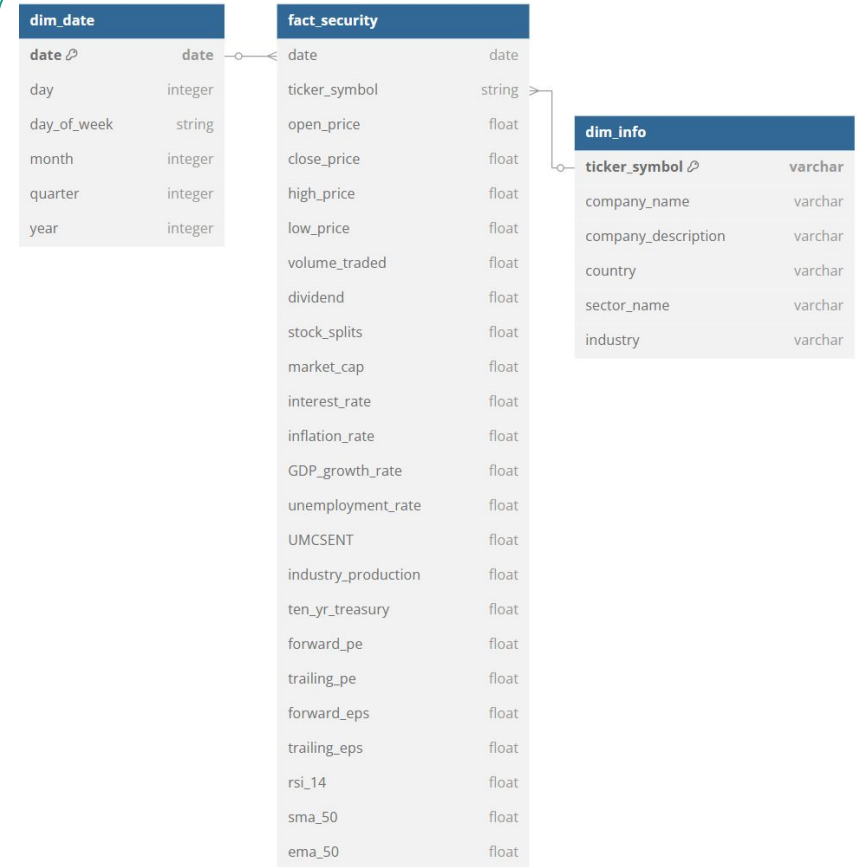
```
TA_undervalue_stock > {} fred_config.json > ...  
1  {  
2    "period": "6y",  
3    "series": {  
4      "CPIAUCSL": "inflation_rate",  
5      "FEDFUNDS": "interest_rate",  
6      "UNRATE": "unemployment_rate",  
7      "GDP": "GDP_growth_rate",  
8      "UMCSENT": "UMCSENT",  
9      "INDPRO": "industrial_production",  
10     "DGS10": "ten_yr_treasury"  
11   }  
12 }  
13
```

Our ingestion process are design with modular in consideration. In this example we created a fred_config.json as an object to be call on the main python code. This allow our program to be able to made modification on ingested dataset easily.

Other Modular JSON file.
stock_config.json for extraction from Yfinance
bq_config.json for login to Big query dataset.

Data Warehouse Design

1. Core Function
 - Tracks financial instruments (stocks/ETFs), their prices, sectors, fundamentals, and macroeconomic links
2. Key Components
 - Master Data: Securities table (core attributes)
 - Market Data: Historical prices
 - Classification: Sector/industry hierarchy
 - Fundamentals: Valuation metrics (P/E, market cap)
 - Macro Context: Economic indicators
3. Supports Analysis Of
 - Performance trends (price history)
 - Sector/industry comparisons
 - Fundamental valuation
 - Macroeconomic correlations
4. Critical Relationships
 - Securities → Multiple prices (1-to-many)
 - Sector → Multiple securities (1-to-many)
 - Security → Fundamentals over time (1-to-many)
5. Design Highlights
 - Uses surrogate keys for stability
 - Time-aware (date fields for temporal analysis)
 - Enforces business rules (e.g., mandatory sector assignment)



Extract, Load, Transform

DBT feature

Key Benefit: Standardized, high-quality, maintainable data workflows.

- **Flexible & Scalable:**
 - Supports various data sources (e.g., CSV files).
 - Modular and reusable models minimize redundancy.
- **SQL-Based Framework:**
 - Simplifies complex workflows using standardized SQL.

```
In [13]: %%bash
          cd stock_analysis
          tree -L 2
```

```
├── README.md
├── analyses
├── dbt_packages
│   ├── dbt_date
│   ├── dbt_expectations
│   └── dbt_utils
├── dbt_project.yml
├── logs
│   └── dbt.log
├── macros
├── models
│   ├── dim_date.sql
│   ├── dim_info.sql
│   ├── fact_security.sql
│   ├── schema.yml
│   ├── schema.yml:Zone.Identifier
│   └── sources.yml
├── package-lock.yml
├── packages.yml
├── profiles.yml
├── schema.yml
├── seeds
├── snapshots
├── target
│   ├── compiled
│   │   ├── graph.gpickle
│   │   ├── graph_summary.json
│   │   ├── manifest.json
│   │   └── partial_parse.msgpack
│   ├── run
│   │   ├── run_results.json
│   │   └── semantic_manifest.json
└── tests
```

15 directories, 19 files

Data Quality

- **Data Quality Assurance:**
 - a. Built-in tests (e.g., unique, not null) ensure reliability during deployments.
- **Leverages Open-Source Packages:**
 - a. Tools like `dbt_utils` simplify validation and transformations.

```
1  version: 2
2
3  models:
4    - name: fact_security
5      description: "Fact table for security"
6      tests:
7        - dbt_expectations.expect_compound_columns_to_be_unique:
8          column_list: ["ticker_symbol", "date"]
9      columns:
10       - name: date
11         description: "The foreign key to the date dimension"
12         tests:
13           - not_null
14           - dbt_utils.accepted_range:
15             max_value: "CURRENT_DATE()"
16           - relationships:
17             to: ref('dim_date')
18             field: date
19           - dbt_expectations.expect_column_values_to_be_of_type:
20             column_type: date
21       - name: ticker_symbol
22         description: "The foreign key to the info dimension"
23         tests:
24           - not_null
25           - relationships:
26             to: ref('dim_info')
27             field: ticker_symbol
28           - dbt_expectations.expect_column_values_to_be_of_type:
29             column_type: string
30       - name: close_price
31         description: "close price of the security"
32         tests:
33           - not_null
34           - dbt_expectations.expect_column_values_to_be_of_type:
35             column_type: float64
36       - name: open_price
37         description: "opening price of the security"
38         tests:
39           - not_null
40           - dbt_expectations.expect_column_values_to_be_of_type:
41             column_type: float64
```


Pipeline Orchestration

- Jobs** **Definition:**
Dagster organizes tasks into defined jobs (upload_yfinance_job, dbt_job, dbt_test, ta_compute). These jobs wrap Python scripts or commands using Dagster's decorators (@job, @op).
- Operations** **(ops):**
Each task within the job, such as executing scripts (yfinance, fred, stock_info) and DBT commands (run_dbt_yfinance, run_dbt_test), is encapsulated within an @op function, ensuring clear modularity and maintainability.
- Scheduling:**
Jobs are scheduled to run automatically using cron schedules. For instance:
 - upload_yfinance_job runs daily at 7:30 AM (SGT).
 - ta_compute executes daily at 7:37 AM.
 - dbt_job is set for daily execution at 7:45 AM.
 - dbt_test executes daily at 8:00 AM.
- Execution & Monitoring:**
Dagster manages execution, handles errors gracefully, and provides logging to easily track and debug job runs.



Pipeline Orchestration

- Jobs** **Definition:**
Dagster organizes tasks into defined jobs (upload_yfinance_job, dbt_job, dbt_test, ta_compute). These jobs wrap Python scripts or commands using Dagster's decorators (@job, @op).
- Operations** **(ops):**
Each task within the job, such as executing scripts (yfinance, fred, stock_info) and DBT commands (run_dbt_yfinance, run_dbt_test), is encapsulated within an @op function, ensuring clear modularity and maintainability.
- Scheduling:**
Jobs are scheduled to run automatically using cron schedules. For instance:
 - upload_yfinance_job runs daily at 7:30 AM (SGT).
 - ta_compute executes daily at 7:37 AM.
 - dbt_job is set for daily execution at 7:45 AM.
 - dbt_test executes daily at 8:00 AM.
- Execution & Monitoring:**
Dagster manages execution, handles errors gracefully, and provides logging to easily track and debug job runs.

```
@job
def upload_yfinance_job():
    run_yfinance_script()
    run_stock_info_script()
    run_fred_script()
```

```
@job
def dbt_job():
    run_dbt_yfinance()
```

```
@job
def ta_compute():
    run_ta_script()
```

```
@job
def dbt_test():
    run_dbt_test()
```

Pipeline Orchestration

- Jobs** **Definition:**
Dagster organizes tasks into defined jobs (upload_yfinance_job, dbt_job, dbt_test, ta_compute). These jobs wrap Python scripts or commands using Dagster's decorators (@job, @op).
- Operations** **(ops):**
Each task within the job, such as executing scripts (yfinance, fred, stock_info) and DBT commands (run_dbt_yfinance, run_dbt_test), is encapsulated within an @op function, ensuring clear modularity and maintainability.
- Scheduling:**
Jobs are scheduled to run automatically using cron schedules. For instance:
 - upload_yfinance_job runs daily at 7:30 AM (SGT).
 - ta_compute executes daily at 7:37 AM.
 - dbt_job is set for daily execution at 7:45 AM.
 - dbt_test executes daily at 8:00 AM.
- Execution & Monitoring:**
Dagster manages execution, handles errors gracefully, and provides logging to easily track and debug job runs.

```
@job
def upload_yfinance_job():
    run_yfinance_script()
    run_stock_info_script()
    run_fred_script()
```

```
@job
def dbt_job():
    run_dbt_yfinance()
```

```
@job
def ta_compute():
    run_ta_script()
```

```
@job
def dbt_test():
    run_dbt_test()
```

Pipeline Orchestration



Applications: Jobs - Google Chrome ingestion_pipe

Jobs

127.0.0.1:3000/jobs

Overview Runs Assets **Jobs** Automation Deployment

Jobs

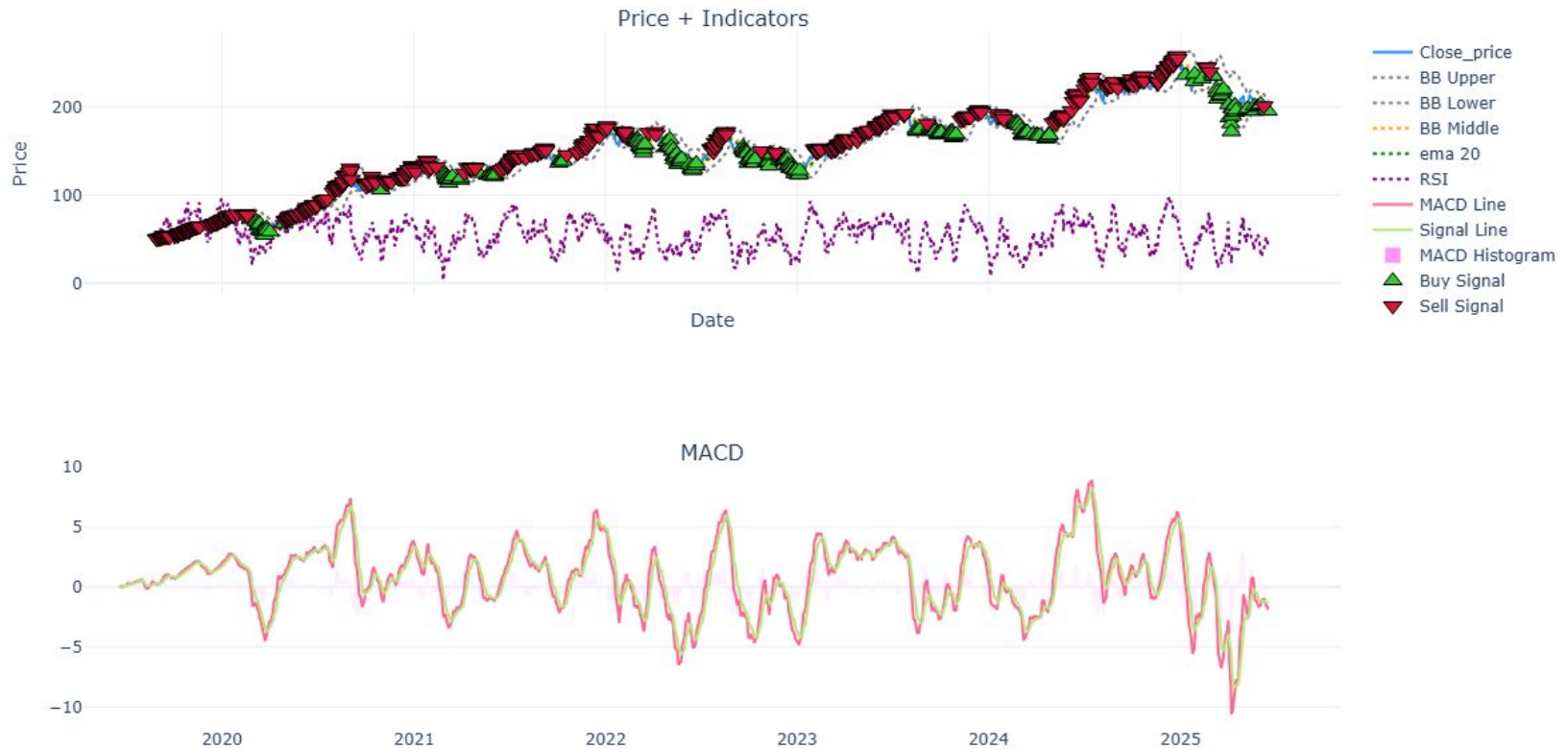
Filter

Name	Schedules/sensors	Latest run	Run history
Ingestion_pipe			
dbt_job	At 07:45 AM GMT+8	Jun 19, 7:45 AM	
dbt_test	At 08:00 AM GMT+8	Jun 19, 8:00 AM	
ta_compute	At 07:37 AM GMT+8	Jun 19, 7:37 AM	
upload_yfinance_job	At 07:30 AM GMT+8	Jun 19, 9:52 AM	



User story 3

AAPL Stock Price with Early Entry Signals



Challenges

Where	Issue	Why	Solution
Dagster Sensor	Sensor failed	Mismatch of package version, package compatibility	
The date column from stock price includes a timestamp.	BigQuery regards date with time information included as a timestamp. This cause issue with other table that only uses Date datatype	Default behaviour	Make use of the DBT Dim table to transform the timestamp to the date datatype.
Dagster Deployment via @assets	difficult to debug and time consuming	code has to be refactored and tested. bugs cannot be easily fixed.	Use a Dagster wrapper and @ops. Code doesn't need to be refactored. Can be directly called from a Py or jupyter notebook. Easier for step-by-step debugging.

Example of the issues

Core Issues Faced:

- `RunRequest.__new__()` got an unexpected keyword argument 'job'
 Cause: Older versions of Dagster don't support `job=` in `RunRequest()`—should use `job_name=` instead.
- `run_status_sensor()` got an unexpected keyword argument 'target'
 Cause: `target=` is not a valid argument in older versions—should be `monitored_jobs=[...]`.
- `NameError: name 'RunStatusSensorContext' is not defined`
 Cause: Missing import for `RunStatusSensorContext`.

Example of a @ Asset code

```
# yfinance_asset.py
import yfinance as yf
import pandas as pd
from dagster import asset, Output

@asset
def fetch_yfinance_data() -> Output[pd.DataFrame]:
    ticker_symbol = "AAPL"
    ticker_data = yf.Ticker(ticker_symbol)
    historical_data = ticker_data.history(period="30d")

    return Output(historical_data)
```

Future works

- Increase the data pool by including other sources that are required to improve the analysis
- Dividend payout date for each stock is different and may require more work to link the code base to tied to the specific ticker dividend date. Alternative includes checking for Non null value in the annual basis/quarterly basis.

Extract, Load, Transform

Limitations of the Current DBT Implementation

Category	Description	Potential Solution
Data Type Validation	Validation of <code>float</code> and <code>integer</code> types using <code>dbt_expectations.expect_column_values_to_be_of_type</code> failed. These tests were commented out.	Investigate root causes, update DBT expectations package, or adjust source schema definitions.
ETF-Specific Test Cases	ETF tickers lack country, sector, and industry metadata, unlike stocks. Some <code>not_null</code> checks were commented out.	Develop separate schema and validation logic for ETF-specific columns.
Regex Validation	No regular expressions were used to validate column patterns (e.g., ticker symbols, ISO country codes).	Add regex-based tests to enforce column-specific format rules.
Range Validation	Did not check max/min values for numeric columns (e.g., stock prices, volume).	Implement boundary checks for numeric data to ensure values fall within realistic thresholds.
Set-Based Validation	Tests to validate column values against an expected list (e.g., valid industry codes) were not implemented.	Use <code>dbt_expectations.expect_column_values_to_be_in_set</code> to enforce compliance with predefined reference data.
General Time Constraints	Some additional validations (e.g., edge cases, hierarchical relationships) were not included due to time limits.	Extend test coverage to include edge cases and domain-specific logic in future iterations.



Agenda

1. Executive summary
2. Tools and Technologies
3. Data Pipeline Architecture
4. Data ingestion
5. Data Warehouse design
6. Extract, Load, and Transform (ELT) & Data Quality
7. Data analysis & Insights
8. Challenges
9. Next Step